

TUGAS PRAKTIKUM 2

SISTEM OPERASI



MUHAMMAD MAULANA NUR SULAIMAN

2410651033

UNIVERITAS MUHAMMDIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

2025

Tugas 1: Sistem Manajemen Proses

1. Buat file program baru:

```
Nano sistem.proses.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <time.h>
void bubble_sort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n-1; i++) {
        for (j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int linear_search(int arr[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (arr[i] == key)
            return i;
    return -1;
}

long long factorial(int n) {
    long long res = 1;
    for (int i = 1; i <= n; i++)
        res *= i;
    return res;
}

double get_time_diff(struct timeval start, struct timeval end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1e6;
```

```

}
int main() {
    pid_t pid;
    int status;
    struct timeval start, end;
    printf("=== Simulasi Sistem Manajemen Proses ===\n\n");

    for (int i = 0; i < 3; i++) {
        gettimeofday(&start, NULL);
        pid = fork();
        if (pid < 0) {
            perror("Fork gagal");
            exit(1);
        }
        else if (pid == 0) { // Child Process
            gettimeofday(&end, NULL);
            srand(time(NULL) ^ (getpid() << 16));
            int arr[10];
            for (int j = 0; j < 10; j++)
                arr[j] = rand() % 100;
            double exec_time;
            gettimeofday(&start, NULL);
            if (i == 0) {
                printf("[Child %d] PID: %d - Tugas: Sorting\n", i+1, getpid());
                bubble_sort(arr, 10);
                printf("[Child %d] Hasil sorting: ", i+1);
                for (int j = 0; j < 10; j++) printf("%d ", arr[j]);
                printf("\n");
            }
            else if (i == 1) {
                printf("[Child %d] PID: %d - Tugas: Searching\n", i+1, getpid());
                int key = arr[5];
                int idx = linear_search(arr, 10, key);
                printf("[Child %d] Mencari %d → ditemukan di indeks %d\n",
i+1, key, idx);
            }
        }
    }
}

```

```

        else if (i == 2) {
            printf("[Child %d] PID: %d - Tugas: Calculation (faktorial)\n",
i+1, getpid());
            int num = 10;
            long long res = factorial(num);
            printf("[Child %d] Faktorial dari %d = %lld\n", i+1, num, res);
        }
        gettimeofday(&end, NULL);
        exec_time = get_time_diff(start, end);
        printf("[Child %d] Execution Time: %.6f detik\n", i+1, exec_time);
        exit(0);
    }
}

// Parent Process
for (int i = 0; i < 3; i++) {
    pid_t cpid = wait(&status);
    if (WIFEXITED(status)) {
        printf("[Parent] Child dengan PID %d selesai dengan status %d\n",
cpid, WEXITSTATUS(status));
    }
}
printf("\n=== Semua proses selesai ===\n");
return 0;
}

```

Program C di atas mensimulasikan sistem manajemen proses dengan menggunakan tiga *child process* (proses turunan) yang masing-masing menjalankan tugas berbeda.

Penjelasan:

1. Parent process membuat 3 child process menggunakan `fork()`.
2. Setiap *child process* menjalankan tugas yang berbeda:
 - Child 1 → Melakukan *bubble sort* pada array acak.
 - Child 2 → Melakukan *linear search* pada array acak.
 - Child 3 → Menghitung *faktorial* dari angka 10.

3. Waktu eksekusi tiap *child* diukur menggunakan `gettimeofday()` dan fungsi `get_time_diff()`.
4. Setelah setiap *child process* selesai, parent process menunggu dengan `wait()` dan menampilkan status selesai tiap proses.
5. Di akhir, parent menampilkan pesan bahwa semua proses telah selesai.

Program ini menunjukkan bagaimana proses dapat berjalan secara paralel menggunakan `fork()`, menjalankan tugas berbeda, mengukur waktu eksekusi, dan disinkronkan kembali oleh *parent process*.

Program ini menunjukkan bagaimana sistem operasi mengelola beberapa proses secara bersamaan menggunakan konsep `fork` dan `wait`. Setiap *child process* dapat menjalankan tugasnya sendiri (sorting, searching, dan perhitungan faktorial) secara independen, sementara *parent process* menunggu semua anak selesai sebelum menutup program. Dengan demikian, program ini membuktikan pentingnya manajemen proses dalam pemrograman sistem untuk melakukan tugas-tugas secara paralel dan efisien.

2. Compile dan jalan
gcc sistem_proses.c -o sistem_proses
./sistem_proses

```
alangenteng@LAPTOP-CV1QG677:~$ nano sistem_proses.c
alangenteng@LAPTOP-CV1QG677:~$ gcc sistem_proses.c -o sistem_proses
alangenteng@LAPTOP-CV1QG677:~$ ./sistem_proses
=== Simulasi Sistem Manajemen Proses ===

[Child 1] PID: 594 - Tugas: Sorting
[Child 1] Hasil sorting: 10 19 23 23 24 38 62 65 68 96
[Child 1] Execution Time: 0.000201 detik
[Child 2] PID: 595 - Tugas: Searching
[Child 2] Mencari 76 → ditemukan di indeks 5
[Child 2] Execution Time: 0.000159 detik
[Parent] Child dengan PID 594 selesai dengan status 0
[Parent] Child dengan PID 595 selesai dengan status 0
[Child 3] PID: 596 - Tugas: Calculation (faktorial)
[Child 3] Faktorial dari 10 = 3628800
[Child 3] Execution Time: 0.000072 detik
[Parent] Child dengan PID 596 selesai dengan status 0

=== Semua proses selesai ===
```

Tugas 2: Chat Application dengan IPC

1. Buat file program baru:
chat_ipc.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h> // O_* constants
#include <sys/mman.h>
#include <sys/stat.h> // mode constants
#include <semaphore.h>
#include <time.h>
#include <errno.h>
#include <sys/types.h>

#define SHM_NAME "/chat_shm_example_rafi"
#define SEM_NAME "/chat_sem_example_rafi"

#define MAX_MESSAGES 200
#define MAX_TEXT_LEN 512
#define MAX_NAME_LEN 64

typedef struct {
    pid_t pid;
    char user[MAX_NAME_LEN];
    time_t ts;
    char text[MAX_TEXT_LEN];
    int valid; // 0 = empty, 1 = used
} chat_msg_t;

typedef struct {
    int head; // index of oldest message
    int tail; // index after newest message
    int count; // number of messages currently stored
    chat_msg_t msgs[MAX_MESSAGES];
} chat_shm_t;

/* helper timestamp -> string */
static void ts_to_str(time_t t, char *buf, size_t n) {
    struct tm tm;
```

```

    localtime_r(&t, &tm);
    strftime(buf, n, "%Y-%m-%d %H:%M:%S", &tm);
}

/* open or create shared memory and semaphore */
static int open_resources(chat_shm_t **out_shm, sem_t **out_sem, int
*created_shm) {
    int shm_fd;
    int first_create = 0;

    shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT | O_EXCL,
0666);
    if (shm_fd >= 0) {
        first_create = 1;
        if (ftruncate(shm_fd, sizeof(chat_shm_t)) == -1) {
            perror("ftruncate");
            close(shm_fd);
            shm_unlink(SHM_NAME);
            return -1;
        }
    } else if (errno == EEXIST) {
        shm_fd = shm_open(SHM_NAME, O_RDWR, 0);
        if (shm_fd < 0) {
            perror("shm_open existing");
            return -1;
        }
    } else {
        perror("shm_open");
        return -1;
    }

    void *map = mmap(NULL, sizeof(chat_shm_t), PROT_READ |
PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (map == MAP_FAILED) {
        perror("mmap");
        close(shm_fd);
        return -1;
    }
    close(shm_fd);

    chat_shm_t *shm = (chat_shm_t *)map;

    if (first_create) {

```

```

    shm->head = 0;
    shm->tail = 0;
    shm->count = 0;
    for (int i = 0; i < MAX_MESSAGES; ++i) shm->msgs[i].valid = 0;
}

sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
if (sem == SEM_FAILED) {
    perror("sem_open");
    munmap(map, sizeof(chat_shm_t));
    return -1;
}

*out_shm = shm;
*out_sem = sem;
if (created_shm) *created_shm = first_create;
return 0;
}

/* write message (protected by semaphore) */
static void write_message(chat_shm_t *shm, sem_t *sem, const char *user,
const char *text) {
    sem_wait(sem);
    int pos = shm->tail;
    chat_msg_t *m = &shm->msgs[pos];
    m->pid = getpid();
    strncpy(m->user, user, MAX_NAME_LEN-1);
    m->user[MAX_NAME_LEN-1] = '\0';
    m->ts = time(NULL);
    strncpy(m->text, text, MAX_TEXT_LEN-1);
    m->text[MAX_TEXT_LEN-1] = '\0';
    m->valid = 1;

    shm->tail = (shm->tail + 1) % MAX_MESSAGES;
    if (shm->count < MAX_MESSAGES) {
        shm->count++;
    } else {
        shm->head = (shm->head + 1) % MAX_MESSAGES; // overwrite
oldest
    }
    sem_post(sem);
}

```



```

/* print history in the requested format */
static void print_history(chat_shm_t *shm, sem_t *sem) {
    sem_wait(sem);
    printf("=== CHAT ROOM ===\n");
    int idx = shm->head;
    for (int i = 0; i < shm->count; ++i) {
        chat_msg_t *m = &shm->msgs[idx];
        if (m->valid) {
            printf("%s> %s\n", m->user, m->text); // format: User1> pesan
        }
        idx = (idx + 1) % MAX_MESSAGES;
    }
    if (shm->count == 0) {
        printf("(no messages)\n");
    }
    printf("=== end ===\n");
    sem_post(sem);
}

```

```

/* cleanup resources */
static void cleanup_resources() {
    if (shm_unlink(SHM_NAME) == -1) {
        if (errno != ENOENT) perror("shm_unlink");
    } else {
        printf("Shared memory removed.\n");
    }
    if (sem_unlink(SEM_NAME) == -1) {
        if (errno != ENOENT) perror("sem_unlink");
    } else {
        printf("Semaphore removed.\n");
    }
}

```

```

int main(int argc, char *argv[]) {
    if (argc == 2 && strcmp(argv[1], "--cleanup") == 0) {
        cleanup_resources();
        return 0;
    }
}

```

```

chat_shm_t *shm;
sem_t *sem;
if (open_resources(&shm, &sem, NULL) != 0) {

```

```

        fprintf(stderr, "Gagal membuka resource shared memory /
semaphore\n");
        return 1;
    }

```

```

    char name[MAX_NAME_LEN];
    printf("Nama Anda: ");
    if (!fgets(name, sizeof(name), stdin)) {
        printf("Input error\n");
        return 1;
    }
    name[strcspn(name, "\n")] = '\0';
    if (strlen(name) == 0) strncpy(name, "anon", MAX_NAME_LEN);

```

```

    printf("Halo %s! Ketik 'help' untuk perintah (send/history/exit).\n",
name);

```

```

    char line[1024];
    while (1) {
        printf("%s> ", name);
        if (!fgets(line, sizeof(line), stdin)) {
            printf("\nEOF. Keluar.\n");
            break;
        }
        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "help") == 0) {
            printf("Perintah:\n");
            printf("  send <pesan> - kirim pesan\n");
            printf("  history      - lihat history chat\n");
            printf("  exit         - keluar\n");
            continue;
        } else if (strncmp(line, "send ", 5) == 0) {
            const char *msg = line + 5;
            if (strlen(msg) == 0) {
                printf("Pesan kosong.\n");
                continue;
            }
            write_message(shm, sem, name, msg);
            continue;
        } else if (strcmp(line, "history") == 0) {
            print_history(shm, sem);
            continue;
        }
    }

```

```

    } else if (strcmp(line, "exit") == 0) {
        printf("Keluar. Bye %s\n", name);
        break;
    } else if (strlen(line) == 0) {
        continue;
    } else {
        printf("Perintah tidak dikenal. Ketik 'help'.\n");
        continue;
    }
}

munmap(shm, sizeof(chat_shm_t));
sem_close(sem);
return 0;
}

```

Program di atas adalah simulasi aplikasi chat berbasis shared memory dan semaphore di Linux (IPC – Inter Process Communication).

Penjelasan:

- 1) Tujuan utama:
Program ini memungkinkan beberapa proses (pengguna) untuk berbagi pesan (chat) secara bersamaan menggunakan shared memory sebagai tempat penyimpanan pesan dan semaphore sebagai pengatur akses agar tidak terjadi bentrok (*race condition*).
- 2) Struktur utama:
 - chat_msg_t menyimpan data tiap pesan (PID, nama pengguna, waktu, teks).
 - chat_shm_t menyimpan daftar pesan (*message buffer*) dan indeks posisi pesan (*head, tail, count*).
- 3) Fungsi penting:
 - open_resources() → Membuka atau membuat *shared memory* dan *semaphore*.
 - write_message() → Menulis pesan baru ke *shared memory* (dilindungi semaphore).
 - print_history() → Menampilkan seluruh riwayat pesan.
 - cleanup_resources() → Menghapus *shared memory* dan *semaphore* (jika sudah tidak digunakan).
- 4) Interaksi pengguna:
Setelah memasukkan nama, pengguna dapat mengetik:
 - send <pesan> → Mengirim pesan ke ruang chat bersama.
 - history → Melihat semua pesan yang sudah dikirim oleh semua pengguna.

- `exit` → Keluar dari aplikasi.
- `--cleanup` (opsional, saat menjalankan program) → Menghapus resource IPC dari sistem.

5) Kesimpulan

singkat:

Program ini memperlihatkan bagaimana beberapa proses bisa berkomunikasi secara real-time melalui *shared memory*, dengan *semaphore* sebagai mekanisme sinkronisasi agar data tetap aman dan konsisten.

Program ini menunjukkan bagaimana *Inter Process Communication (IPC)* dapat digunakan untuk membuat sistem chat sederhana di Linux dengan memanfaatkan *shared memory* sebagai media penyimpanan pesan bersama dan *semaphore* untuk menjaga sinkronisasi antar proses. Melalui mekanisme ini, beberapa proses dapat saling bertukar pesan secara real-time tanpa saling mengganggu data satu sama lain. Program ini juga menggambarkan pentingnya pengelolaan sumber daya bersama serta penerapan konsep *concurrency control* dalam sistem operasi.

2. Compile dan jalan

```
gcc -o chat_ipc chat_ipc.c -lrt -pthread
./chat_ipc
```

The image shows two terminal windows side-by-side. The left window shows the compilation of `chat_ipc.c` into `chat_ipc` using `gcc -o chat_ipc chat_ipc.c -lrt -pthread`, followed by running `./chat_ipc`. The right window shows the execution of the program, where two users, 'alan' and 'rafi', interact in a chat room. The chat interface includes a help menu with options like 'send', 'history', 'exit', and 'send <pesan>'.

```

alanteng@LAPTOP-CV1QG677:~$ nano chat_ipc.c
alanteng@LAPTOP-CV1QG677:~$ gcc -o chat_ipc chat_ipc.c -lrt -pthread
alanteng@LAPTOP-CV1QG677:~$ ./chat_ipc
Nama Anda: alan
Halo alan! Ketik 'help' untuk perintah (send/history/exit).
alan> help
Perintah:
  send <pesan> - kirim pesan
  history      - lihat history chat
  exit        - keluar
alan> send <halo fi>
alan> send <gimana kabarnya>
alan> history
=== CHAT ROOM ===
alan> <hello>
rafi> <hai juga>
alan> <halo fi>
rafi> <halo juga lan>
alan> <gimana kabarnya>
rafi> <alhamdulillah baik>
=== end ===
alan> exit
Keluar. Bye alan
alanteng@LAPTOP-CV1QG677:~$

alanteng@LAPTOP-CV1QG677:~$ ./chat_ipc
Nama Anda: rafi
Halo rafi! Ketik 'help' untuk perintah (send/history/exit).
rafi> help
Perintah:
  send <pesan> - kirim pesan
  history      - lihat history chat
  exit        - keluar
rafi> send <halo juga lan>
rafi> send <alhamdulillah baik>
rafi> history
=== CHAT ROOM ===
alan> <hello>
rafi> <hai juga>
alan> <halo fi>
rafi> <halo juga lan>
alan> <gimana kabarnya>
rafi> <alhamdulillah baik>
=== end ===
rafi> exit
Keluar. Bye rafi
alanteng@LAPTOP-CV1QG677:~$
```